

國立臺南大學 資訊工程學系

計算機程式設計(二) C++ Template 自主練習作業 第 2 週小專題報告



指導教授：林朝興教授

系所：資訊工程學系一年級

學號：S11459041

姓名：邱守謙

日期：西元 2026年06月05日

Link: https://github.com/s11459041/-_Student-Score-Management-System

一、自主學習主題與目標：

本次作業的學習主題為「C++ STL 容器操作與 Template 程式設計」，目標是透過實作一個完整的學生成績管理系統，深入理解 STL 的使用方式 Template Function 的設計邏輯，熟悉常用 STL 容器，並理解這些容器背後對應的基礎資料結構。具體學習目標包含：

1. 理解 template 與 STL 的關係。
2. 會使用 vector、stack、queue、priority_queue、set、map 等常見容器。
3. 能依問題需求選擇合適的資料結構。
4. 能撰寫可在 g++ -std=c++17 環境下編譯執行的程式。
5. 為大二資料結構課程建立先備基礎。

二、Template 與 STL 的關係：

C++ STL (Standard Template Library) 本身即大量運用 Template 技術建構，例如：`vector<int>`、`map<string, Student>` 中的尖括號型別參數，正是 Template 的具體表現。Template 允許我們撰寫與型別無關的通用程式碼，編譯器在遇到具體型別時自動實例化對應的函式或類別。STL 的容器 (container)、演算法 (algorithm)、疊代器 (iterator) 三大元件均以 Template 為基礎實作：

- 容器：`vector<T>`、`map<K,V>`、`stack<T>`、`queue<T>` 皆為 template class。
- 演算法：`sort()`、`find()`、`accumulate()` 等皆為 template function，可作用於不同型別的容器。
- 疊代器：作為容器與演算法之間的橋樑，亦以 template 定義。

本次小專題透過自定義 `getMax<T>` 與 `getMin<T>` 兩個 template function，實際體驗了與 STL 相同的設計思想。

三、常見 STL 容器用途整理：

1. `vector<T>` - 動態陣列：

- 特性：可以動態調整大小，無需事先宣告大小，可根據資料量自動擴充或縮小容量。隨機存取 $O(1)$ 、尾端新增 $O(1)$ ，適合需要排序或索引的場合。
- 本專題用途：將 map 資料轉換為 `vector<Student>` 後，使用 `sort()` 依成績排序。

2. `map<K, V>` - 有序鍵值對：

- 特性：依 key 自動排序 (紅黑樹)，查找 $O(\log n)$ ，適合以唯一鍵快速存取資料。
- 本專題用途：以學號 (string) 為 key，儲存完整學生資料 (Student struct)。

3. `stack<T>` - 堆疊(LIFO):

- 特性:後進先出, 僅操作頂端元素。
- 練習題用途:實作第二題的括號配對檢查, 遇到左括號 push, 遇到右括號比對後 pop。

4. `queue<T>` - 佇列(FIFO):

- 特性:先進先出, 從尾端加入、從前端取出。
- 練習題用途:實作第三題的模擬服務台排隊, 依序服務最前面的顧客。

四、小專題(第2週)設計說明:

1. 資料結構設計:

本程式使用`struct Student` 儲存每筆資料, 並以`map<string, Student>` 作為主要容器, 學號為key:

```
struct Student {  
    string id;  
    string name;  
    int score;  
};
```

```
map<string, Student> students; // 以學號為 key
```

選擇map 的原因:學號具唯一性, 適合作為key;map 內建排序, 列出時自動依學號排列;查詢速度 $O(\log n)$, 比線性搜尋vector 更有效率。

2. 五大功能說明:

- 功能 1 - 新增學生:輸入學號、姓名、成績, 並以 `students[id] = s` 存入 map。若學號已存在, 詢問是否覆蓋, 避免誤刪資料。
- 功能 2 - 列出所有學生:使用range-based for loop 走訪 map, 以`setw()` 格式化輸出學號、姓名、成績。
- 功能 3 - 依成績排序:將map 轉為 `vector<Student>`, 使用`sort()` 來由高到低排序, 並顯示名次。
- 功能 4 - 查詢學生:以`students.count(id)` 判斷學號是否存在, 找到則顯示資料, 否則提示錯誤訊息。
- 功能 5 - 統計成績:遍歷所有學生, 呼叫 `getMax()` 與 `getMin()` 的template function 計算最高最低分, 另計算平均、及格人數與不及格人數。

五、主要程式架構與重要程式片段:

1. Template Function:

定義兩個通用 template function, 可比較任何支援 > 運算子的型別:

```
template <class T>  
T getMax(T a, T b) {
```

```

        return (a > b) ? a : b;
    }

    template <class T>
    T getMin(T a, T b) {
        return (a < b) ? a : b;
    }

```

實際呼叫位置: 功能 5「統計成績」中, 傳入 int 型別的成績進行比較:

```

int maxScore = v[0].score;
int minScore = v[0].score;
for (auto& s : v){
    //Use template function(template is automatically deduced as int)
    maxScore = getMax(maxScore,s.score);
    minScore = getMin(minScore,s.score);
}

```

2. 功能 3 - 依成績排序sort():

```

vector<Student> v;
for(auto& p : students){
    v.push_back(p.second);
}
sort(v.begin(), v.end(), [](const Student& a, const Student& b){
    return a.score > b.score;
});

```

3. 功能 2、3、5 - 在還沒任何學生時, 提示還未有任何學生, 並回到選單:

```

if(students.empty()){
    cout << "No student right now" << endl;
    cout << endl;
    return;
}

```

4. 主選單迴圈:

```

int main(){
    int choice;
    do {
        cout << "----Student Score Management System----" << endl;
        cout << "1. Add student" << endl;
        // .....
        cout << "0. Exit" << endl;
        cout << endl << "Enter the option you want:" << endl;
        cin >> choice;
        cout << endl;
        switch (choice){
            case 1: addStudent();
                    break;
            case 2: printAllStuInfo();
                    break;
            // .....
        }
    }while(choice != 0);
    return 0;
}

```

六、執行畫面或輸出結果:

以下為實際執行程式後的輸出範例：

- 輸入1 並新增一位學生(以Bobby 為例):

```
----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter the option you want:
1

Enter student id:
s001
Enter student name:
Bobby
Enter student score:
85
Student Bobby added successfully!!
```

- 輸入1 並重複新增一位學生(Bobby), 測試若學號重複, 自行決定覆蓋:

```
---Student Score Management System---
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter the option you want:
1

Enter student id:
s001
Enter student name:
Bobby
Enter student score:
75
This student id already exist, do you want to overwrite it(y/n)
y
Overwrite it successfully!!
Student Bobby added successfully!!
```

- 輸入4 搜尋一位學生(Bobby), 並確定結果是否為前面覆蓋後的(成績85->75):

```

----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter the option you want:
4

Enter the student ID you want to find:
s001

ID: s001 Name: Bobby Score: 75

```

- 再新增了四位學生後(Alice, Jus, Kevin, Timmy), 輸入2, 查看所有的學生:

```

----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter the option you want:
2

Id          Name      Score
-----
s001        Bobby      75
s002        Alice      48
s003        Jus        97
s004        Kevin      62
s005        Timmy      52

```

- 輸入3, 根據成績排列:

```

----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter the option you want:
3

Sort by the score
RankedId    Name      Score
-----
1   s003     Jus       97
2   s001     Bobby     75
3   s004     Kevin     62
4   s005     Timmy     52
5   s002     Alice     48

```

- 輸入5, 查看最高低分、及格/不及格人數跟平均分數:

```
----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

Enter the option you want:
5

The average score:66.8
The highest score:97
The lowest score:48
The number of people who passed:3
The number of people who didn't pass:2
```

- 嘗試在沒有學生加入的情況下，分別輸入2、3、5:

```
Enter the option you want:
2

No student right now

----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
```

```
Enter the option you want:
3

No student right now

----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
```

```
Enter the option you want:
5

No student right now

----Student Score Management System----
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
```

七、GitHub repo link與檔案結構說明:

GitHub Repository (Public):

https://github.com/s11459041/-_Student-Score-Management-System

八、遇到的問題與解決方式:

1. 問題一: 不了解auto 的作用與替代寫法。

- 解決方式: 在學習STL 容器時, 許多範例程式中大量使用auto 關鍵字, 但因為這個我東西沒有用過, 所以最初只是照著寫, 導致對那段程式碼的理解停留在表面, 時常寫道那裡的時候就卡關, 所以為此也查了不少資料, 最後才理解到auto 是讓編譯器自動推導型別的語法, map<string, Student> 的每個元素本質上是 pair<string, Student>, 因此用p.first 可以取得學號(key), 用p.second 可以取得學生資料(value)。了解這個底層型別之後, 再回頭看 \auto 的寫法, 反而覺得更清晰, 因為知道它在簡化的是什麼。此後在程式中仍選擇使用auto, 因為已經能夠確定它推導出來的型別, 不再是不明所以地使用。

2. 問題二: map 無法直接使用 sort()

- 解決方式: 這個也是在作業說明就有提到的, 若使用 map 作為主要儲存, 可另外轉成vector 後排序。但本質上我不知道為什麼, 因為前面練習其實只是知道要那樣用, 但不理解兩者間的底層邏輯, 而在更詳盡的查找後, 了解到sort() 需要 random access iterator(隨機存取疊代器), 才能在 $O(n \log n)$ 的時間內完成排序; 但map 的底層結構是紅黑樹(Red-Black Tree), 其疊代器屬於 bidirectional iterator(雙向迭代器), 只能一步一步向前或向後移動, 無法跳躍存取, 因此不符合 sort() 的需求。所以也讓我了解到選擇容器不只是看「能不能存資料」, 還要考慮「之後需要對資料做什麼操作」。如果主要需求是快速查找, 選map; 如果需要頻繁排序, 選vector 可能更合適, 或者像本程式一樣, 以map 為主要儲存、需要排序時再轉換為vector, 兩者各取所長。

九、學習心得與未來可延伸方向:

透過本次的小專題, 我對STL 容器的使用有了更具體的理解。以往我大概只知道 vector 和 array 的差別, 現在能夠根據資料特性選擇最適合的容器: 需要快速查找 map、需要排序用 vector + sort() 等。Template Function 的設計讓我理解了「一次撰寫, 多型別適用」的概念, 這也是 STL 本身的設計哲學。getMax<T> 不需要為 int、double、string 各寫一版, 充分展現了泛型程式設計的優勢。

未來可延伸的方向：

- 將資料讀寫擴充為檔案 I/O, 使程式重啟後仍保留資料(使用 fstream)
- 新增刪除學生功能(map::erase())
- 使用multimap 支援同名學生, 或加入更多欄位(科目成績)
- 將功能封裝為 class, 練習更完整的物件導向設計
- 以Template Class 設計通用排行榜容器, 進一步延伸 template 應用